The Aether Product Handbook

A complete reference for Aether's architecture, agents, retrieval configuration and policies.

1. What is Aether?

Aether is a personal AI workplace that combines document understanding, live web research, and automatic report writing in a single chat interface.

The product was designed as a capstone-level showcase of applied Retrieval-Augmented Generation (RAG) and multi-agent orchestration.

2. The Multi-Agent System

Aether routes every request through a Planner Agent that selects specialised tools.

The Document Analyst Agent answers questions strictly from an uploaded PDF using RAG.

The Researcher Agent performs live web search using the Tavily Search API with an advanced search depth.

The Writer Agent converts the Planner's collected findings into a structured, professional report.

3. Retrieval Pipeline

Uploaded PDFs are parsed page-by-page with pypdf, preserving the page number of every chunk.

Text is split with RecursiveCharacterTextSplitter using a chunk size of 1000 characters and an overlap of 150.

Chunks are embedded with the sentence-transformers model all-MiniLM-L6-v2 and stored in a FAISS index.

At question time, the top 4 chunks (RETRIEVAL_K = 4) are inserted into the Document Analyst prompt.

4. Model Configuration

Aether's default chat model is openai/gpt-oss-20b, served through the Groq API.

The planner temperature is set to 0.7, allowing natural planning while keeping tool use focused.

Every tool call is wrapped by an error-handling middleware so one failing tool never crashes a session.

5. Usage Policies

The Document Analyst must refuse to answer from outside knowledge and must say when a fact is absent from the document.

Users may ask unlimited questions in a session; only one PDF is kept actively indexed at a time.

Chat history is kept in memory only and cleared when the browser tab is refreshed or the Clear Chat button is used.

6. Limits and Roadmap

Scanned PDFs without an extractable text layer are not supported yet; OCR support is on the roadmap.

Planned milestones: memory across sessions, export to PDF, streaming responses, and a plugin system for custom agents.

Section 7. Design Decisions

Design note 1. Why a Planner-Executor rather than a single chain?

A single fixed chain cannot decide between document facts and live web facts. Routing that decision to a planner turns Aether into a system that adapts to the request instead of a pipeline that always does the same thing.

Design note 2. Why keep every chunk's page number?

Answers without provenance are hard to trust. Because metadata survives the split, the Writer Agent can cite pages, and the evaluation harness can measure where answers come from.

Design note 3. Why FAISS rather than a hosted vector database?

The FAQ workload is one document per session. An in-memory FAISS index starts in milliseconds, costs nothing to host, and is automatically discarded with the session - exactly the lifetime the product promises.

Design note 4. Why one LLM for planning and writing but a dedicated chain for the analyst?

The Document Analyst has stricter rules: refuse outside knowledge, cite pages. Isolating its prompt keeps the strictness out of the planner's tool loop, where rigidity would hurt

routing.

Design note 5. Why wrap tool calls in middleware?

An agent session is a multi-step program; one failing HTTP call should not destroy an ongoing analysis. The middleware converts every exception into a ToolMessage the planner can reason about and recover from.

Design note 6. Why a Writer step at all?

Raw tool output mixes formats, urls and partial sentences. A dedicated formatting step guarantees consistent headings, bullets and tables regardless of which tools ran.

Section 8. Troubleshooting and Support

Symptom 1. The app reports no extractable text found.

The PDF is likely a scan or image export. Aether requires a text layer; OCR support is planned in a future milestone. Re-export your file from the original application.

Symptom 2. Answers quote the web instead of my PDF.

The planner routes by the wording of the question. Name the document explicitly, for example: 'According to the handbook, ...'.

Symptom 3. A tool error appears in the middle of an answer.

The middleware caught an upstream failure (search API, quota or network). The planner received the error message and continued with the remaining tools; retrying usually fixes it.

Symptom 4. Vector store errors on startup.

Verify FAISS is installed: faiss-cpu must match your Python version. The index is rebuilt automatically after every upload, so clearing the session also clears a corrupted index.

Symptom 5. Slow embedding on first upload.

The first run downloads the all-MiniLM-L6-v2 weights. They are then cached locally, and embedding a normal document takes seconds.

Symptom 6. Where do I report issues?

Open an issue on the repository with the document title and the exact question that failed. The evaluation harness can be extended with your case.

Appendix A. Configuration Cheat Sheet

Chunk size: 1000 characters. Overlap: 150. Retrieval: top-4. Embeddings: all-MiniLM-L6-v2. Vector store: FAISS. PDF parser: pypdf. Default model: openai/gpt-oss-20b via Groq. Temperature: 0.7. Search: Tavily advanced. Agents: Planner, Document Analyst, Researcher, Writer. Unsupported: scanned PDFs (OCR planned). Planner selects tools; Writer formats the final report.

Appendix B. Frequently Asked Questions

FAQ 1. How many agents does Aether use?

Four: Planner, Document Analyst, Researcher and Writer.

FAQ 2. Can Aether compare two PDFs at once?

No. Only one PDF is actively indexed at any time.

FAQ 3. Does the Document Analyst use outside knowledge?

No. It answers only from the retrieved document context and refuses otherwise.

FAQ 4. What happens on a tool failure?

The tool-call middleware returns an error ToolMessage so the session survives.

FAQ 5. Which API key variables are needed?

GROQ_API_KEY and TAVILY_API_KEY.

FAQ 6. Is Aether free to run?

The demo tier of Groq and Tavily is enough for normal usage.

FAQ 7. Does Aether keep my documents?

No. The FAISS index lives in in-memory session state and disappears when the tab closes.

FAQ 8. What is the embedding dimension of all-MiniLM-L6-v2?

384.

FAQ 9. What does RecursiveCharacterTextSplitter do?

It splits text on paragraphs, then lines, then words until chunks fit the size limit.

FAQ 10. Why keep a chunk overlap of 150?

So facts near chunk boundaries are sentence complete in at least one chunk.

FAQ 11. Can the Researcher and Document Analyst run in the same turn?

Yes. The Planner can call both when a request mixes PDF facts and fresh web context.

FAQ 12. What UI framework powers Aether?

Streamlit, with a fixed bottom chat bar built from st.form and custom CSS.

FAQ 13. What is a Planner-Executor pattern?

One LLM decides which tools to call, specialist tools execute, and a Writer formats the result.

FAQ 14. How are page numbers preserved?

Every chunk inherits the page number from the page it was extracted from.

FAQ 15. What is FAISS?

Facebook AI Similarity Search, an in-memory vector index for nearest-neighbour lookup.

FAQ 16. What temperature should I use for strict document answering?

Lower it closer to 0; Aether uses 0.7 by default for natural planning.

FAQ 17. Does Aether stream responses?

Not yet; streaming is on the roadmap.

FAQ 18. Which Python version is required?

Python 3.10 or later.

FAQ 19. What happens if I upload a scanned PDF?

The app reports no extractable text found; OCR support is planned.

FAQ 20. Are the evaluation scripts included?

Yes. The evaluation folder contains a golden-QA harness with retrieval and LLM-judged metrics.

Appendix C. Glossary

1. Agent - An LLM given a role, tools and rules that decides and executes actions.

2. RAG - Retrieval-Augmented Generation: fetch relevant document chunks, then generate the answer grounded in them.

3. Embedding - A numeric vector representing text meaning; similar texts have nearby vectors.

4. Vector index - A structure that stores embeddings and returns nearest neighbours fast.

5. Chunk - A size-limited slice of document text with metadata.

6. Hit@k - Whether the correct answer text appears among the top k retrieved chunks.

7. MRR - Mean Reciprocal Rank: average of 1 divided by the rank of the first correct hit.

8. Groundedness - Property of an answer whose every claim is supported by the given context.

9. Hallucination - An unsupported claim produced by a model.

10. Gatekeeping - Refusing to answer when the context does not contain the answer.

11. Routing - Choosing which specialist tool/agent should handle a request.

12. Middleware - Code wrapped around every tool call, used in Aether for resilient error handling.

13. Tool - A typed Python function the model can choose to call.

14. Session state - Per-user, in-memory storage Streamlit provides between reruns.

Thank you for reading the handbook. Version 1.0.